$$Nmax = \sqrt{\frac{(1-\alpha)}{\beta}}$$

The resulting capacity gain is relative to the minimum throughput, so the absolute system capacity is obtained as follows:

$$Xmax = X\,(1) \times C\,(Nmax)$$

1.2 Database connections boundaries

Every connection requires a TCP socket from the client (application) to the server (database).

The total number of connections offered by a database server depends on the underlying hardware resources, and finding how many connections a server can handle is possible through measurements and proven scalability models.

> SQL Server 2016[a] and MySQL 5.7[b] use thread-based connection handling.
>
> PostgreSQL 9.5[c] uses one operating system process for each individual connection.
>
> On Windows systems, Oracle uses threads, while on Linux, it uses process-based connections. Oracle 12c[d] comes with a thread-based connection model for Linux systems too.
>
> [a] https://msdn.microsoft.com/en-us/library/ms190219.aspx
> [b] https://dev.mysql.com/doc/refman/5.7/en/connection-threads.html
> [c] http://www.postgresql.org/docs/current/static/connect-estab.html
> [d] http://docs.oracle.com/database/121/CNCPT/process.htm

A look into database system internals reveals the tight dependency on CPU, Memory, and Disk resources. Because I/O operations are costly, the database uses a buffer pool to map into memory the underlying data and index pages. Changes are first applied in memory and flushed to disk in batches to achieve better write performance.

Even if all indexes are entirely cached in memory, disk access might still occur if the requested data blocks are not cached into the memory buffer pool. Not just queries may generate I/O traffic, but the transaction and the redo logs require flushing in-memory data structures periodically so that durability is not compromised.

To provide data integrity, any relational database system must use exclusive locks to protect data blocks (rows and indexes) from being updated concurrently. This is true even if the database system uses MVCC (Multi-Version Concurrency Control) because otherwise atomicity would be compromised. This topic is going to be discussed in greater detail in the Transactions chapter.

This means that high-throughput database applications experience contention on CPU, Memory, Disk, and Locks. When all the database server resources are in use, adding more workload only increases contention, therefore lowering throughput.

Resources might get saturated due to improper system configuration, so the first step to improving a system throughput is to tune it according to the current data access patterns.

Lowering response time not only makes the application more responsive, but it can also increase throughput.

However, response time alone is not sufficient in a highly concurrent environment. To maintain a fixed upper bound response time, the system capacity must increase relative to the incoming request throughput. Adding more resources can improve scalability up to a certain point, beyond which the capacity gain starts dropping.

At the Velocity conference[a], both Google Search and Microsoft Bing teams have concluded that higher response times can escalate and even impact the business metrics.

Capacity planning is a feedback-driven mechanism, and it requires constant application monitoring, and so, any optimization must be reinforced by application performance metrics.

[a] http://radar.oreilly.com/2009/06/bing-and-google-agree-slow-pag.html

1.3 Scaling up and scaling out

Scaling is the effect of increasing capacity by adding more resources. Scaling vertically (scaling up) means adding resources to a single machine. Increasing the number of available machines is called horizontal scaling (scaling out).

Traditionally, adding more hardware resources to a database server has been the preferred way of increasing database capacity. Relational databases have emerged in the late seventies, and, for two and a half decades, the database vendors took advantage of the hardware advancements following the trends in Moore's Law.

Distributed systems are much more complex to manage than centralized ones, and that is why horizontal scaling is more challenging than scaling vertically. On the other hand, for the same price of a dedicated high-performance server, one could buy multiple commodity machines whose sum of available resources (CPU, Memory, Disk Storage) is greater than of the single dedicated server. When deciding which scaling method is better suited for a given enterprise system, one must take into account both the price (hardware and licenses) and the inherent developing and operational costs.

Being built on top of many open source projects (e.g. PHP, MySQL), Facebook[3] uses a horizontal scaling architecture to accommodate its massive amounts of traffic.

StackOverflow[4] is the best example of a vertical scaling architecture. In one of his blog posts[5], Jeff Atwood explained that the price of Windows and SQL Server licenses was one of the reasons for not choosing a horizontal scaling approach.

No matter how powerful it might be, one dedicated server is still a single point of failure, and throughput drops to zero if the system is no longer available. For this reason, database replication is not optional in many enterprise systems.

1.3.1 Master-Slave replication

For enterprise systems where the read/write ratio is high, a Master-Slave replication scheme is suitable for increasing availability.

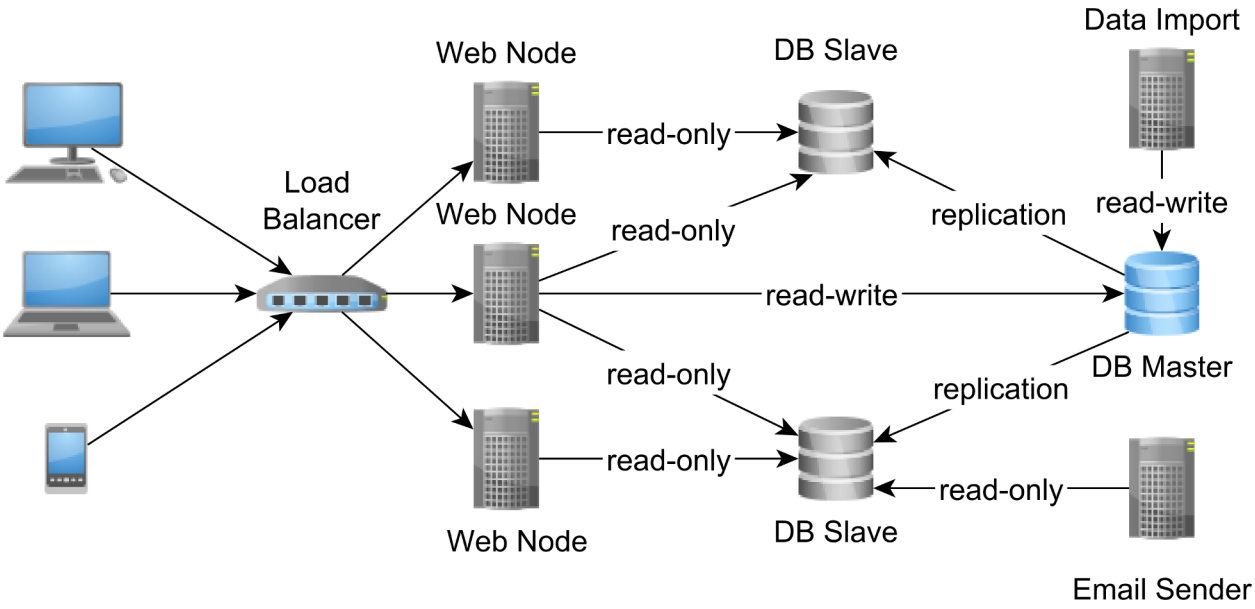

Figure 1.2: Master-Slave replication

The Master is the system of record and the only node accepting writes. All changes recorded by the Master node are replayed onto Slaves as well. A binary replication uses the Master node WAL (Write Ahead Log) while a statement-based replication replays on the Slave machines the exact statements executed on Master.

Asynchronous replication is very common, especially when there are more Slave nodes to update.

[3] https://www.facebook.com/note.php?note_id=409881258919
[4] http://stackexchange.com/performance
[5] http://blog.codinghorror.com/scaling-up-vs-scaling-out-hidden-costs/

The Slave nodes are eventual consistent as they might lag behind the Master. In case the Master node crashes, a cluster-wide voting process must elect the new Master (usually the node with the most recent update record) from the list of all available Slaves.

The asynchronous replication topology is also referred as *warm standby* because the election process does not happen instantaneously.

Most database systems allow one synchronous Slave node, at the price of increasing transaction response time (the Master has to block waiting for the synchronous Slave node to acknowledge the update). In case of Master node failure, the automatic failover mechanism can promote the synchronous Slave node to become the new Master.

Having one synchronous Slave allows the system to ensure data consistency in case of Master node failures since the synchronous Slave is an exact copy of the Master. The synchronous Master-Slave replication is also called a *hot standby* topology because the synchronous Slave is readily available for replacing the Master node.

When only asynchronous Slave nodes are available, the newly elected Slave node might lag behind the failed Master, in which case consistency and durability are traded for lower latencies and higher throughput.

Aside from eliminating the single point of failure, database replication can also increase transaction throughput without affecting response time. In a Master-Slave topology, the Slave nodes can accept read-only transactions, therefore routing read traffic away from the Master node.

The Slave nodes increase the available read-only connections and reduce Master node resource contention, which, in turn, can also lower read-write transaction response time. If the Master node can no longer keep up with the ever-increasing read-write traffic, a Multi-Master replication might be a better alternative.

1.3.2 Multi-Master replication

In a Multi-Master replication scheme, all nodes are equal and can accept both read-only and read-write transactions. Splitting the load among multiple nodes can only increase transaction throughput and reduce response time as well.

However, because distributed systems are all about trade-offs, ensuring data consistency is challenging in a Multi-Master replication scheme because there is no longer a single source of truth. The same data can be modified concurrently on separate nodes, so there is a possibility of conflicting updates. The replication scheme can either avoid conflicts or it can detect them and apply an automatic conflict resolution algorithm.

To avoid conflicts, the two-phase commit protocol can be used to enlist all participating nodes in one distributed transaction. This design allows all nodes to be in sync at all time, at the cost of increasing transaction response time (by slowing down write operations).

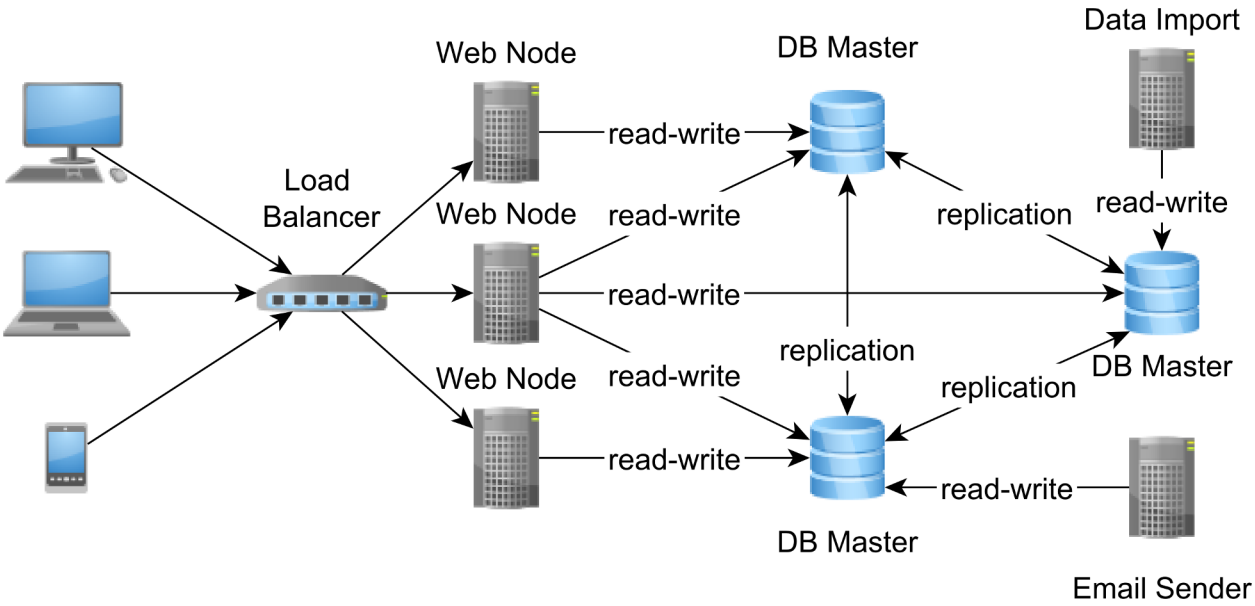

Figure 1.3: Multi-Master replication

If nodes are separated by a WAN (Wide Area Network), synchronization latencies may increase dramatically. If one node is no longer reachable, the synchronization will fail, and the transaction will roll back on all Masters.

Although avoiding conflicts is better from a data consistency perspective, synchronous replication might incur high transaction response times. On the other hand, at the price of having to resolve update conflicts, asynchronous replication can provide better throughput,

The asynchronous Multi-Master replication requires a conflict detection and an automatic conflict resolution algorithm. When a conflict is detected, the automatic resolution tries to merge the two conflicting branches, and, in case it fails, manual intervention is required.

1.3.3 Sharding

When data size grows beyond the overall capacity of a replicated multi-node environment, splitting data becomes unavoidable. Sharding means distributing data across multiple nodes, so each instance contains only a subset of the overall data.

Traditionally, relational databases have offered *horizontal partitioning* to distribute data across multiple tables within the same database server. As opposed to *horizontal partitioning*, sharding requires a distributed system topology so that data is spread across multiple machines.

Each shard must be self-contained because a user transaction can only use data from within a single shard. Joining across shards is usually prohibited because the cost of distributed locking and the networking overhead would lead to long transaction response times.

By reducing data size per node, indexes also require less space, and they can better fit into main memory. With less data to query, the transaction response time can also get shorter too.